Eventual Coherence: Preserving Causal Validity in Replicated Systems

Solving the Problem of Divergent Intent in Shared State Networks

Forest Mars
Principal Architect
mars@mlops.nyc

June 23, 2026

ABSTRACT

The delete-update conflict, where one device deletes a resource while another concurrently updates it, is a fundamental underdetermined challenge in multi-device synchronization. Existing approaches treat it as a synchronization problem rather than a semantics problem and so sacrifice at least one of three properties: availability through coordination, causal validity through last-write-wins semantics, or storage boundedness through conflict-free replicated data types. We argue that this tradeoff is unnecessary and arises from a category error: treating topology operations (CREATE, DELETE) and content operations (UPDATE) as equivalent writes subject to uniform reconciliation semantics. We introduce eventual coherence, a synchronization model grounded in the categorical distinction between these operation classes. Control plane operations establish resource topology and are reconciled at synchronization boundaries. Data plane operations modify resource content and propagate in near-real-time. Reconciliation is governed by two principles. First, local deletion finality: a device that has deleted a resource permanently ignores all subsequent updates to it. Second, server-level upsert semantics: a server receiving a data plane update for a deleted resource restores it, ensuring devices with active causal participation continue receiving updates. Together these principles resolve the delete-update conflict without coordination, without tombstones, and without violating causal validity for any participating device. The formal contribution of this paper is the rejection of the classical consistency hierarchy as the sovereign framework for evaluating human-facing distributed systems. We argue that optimizing for uniform cross-replica agreement represents a framework error when devices act as causal proxies for human agents. We introduce eventual coherence, a correctness criterion that decouples system evaluation into two orthogonal dimensions: inter-replica agreement and local historical validity. The protocol is implementable over standard HTTP without vector clocks, operational transformation, or physical clock synchronization assumptions.

1. INTRODUCTION

Distributed systems that maintain replicated state across multiple devices face a persistent challenge: how to reconcile operations when devices independently modify shared resources [1, 2]. This problem becomes particularly acute when one device deletes a resource while another concurrently updates it. Consider a conversation management system where Device A deletes a conversation, while Device B, operating without knowledge of this deletion, continues adding messages to it. When these devices synchronize, the system must decide: should the deletion propagate, discarding Device B's work, or should the update propagate, implicitly reversing the deletion?

Neither outcome is correct. Device A made a deliberate choice to delete. Device B made a deliberate choice to continue working. A system that erases either choice in the name of convergence has not achieved correctness. It has imposed an arbitrary resolution on a situation that does not require one.

Traditional approaches to this problem fall into three categories, each sacrificing at least one property the other two preserve. Coordination-based systems achieve consistency by requiring devices to obtain locks [3] or reach consensus [4, 5, 6] before performing operations, but this approach sacrifices availability when devices cannot communicate [7], precisely the conditions under which multi-device systems are most valuable [8, 9]. Last-write-wins (LWW) strategies avoid coordination by imposing total ordering based on timestamps [10, 11], but this discards causal validity: a deletion performed after an update may be overridden by that update due to clock skew [13], or an update performed after a deletion may be discarded when the deletion carries a later timestamp, losing work performed in good faith. Conflict-free replicated data types [14, 15, 16] preserve all operations through carefully designed merge semantics, but make true deletion difficult, requiring either tombstones that accumulate without bound [21, 76] or monotonic state growth [20] that prevents genuine removal.

All of these approaches introduce substantial implementation complexity, forcing application developers to reason about specialized data structures, merge semantics, and long-term state management concerns that arise primarily from the synchronization mechanism itself rather than the underlying application domain.

We observe that this problem arises from a category error that runs through all three approaches: treating all operations as equivalent writes subject to uniform reconciliation semantics. Resource creation and deletion establish the topology of the system, what resources exist at all. Resource updates modify the content of existing resources without changing topology. These are different kinds of operation having different causal properties [12, 22], different timing requirements, and different correct reconciliation behaviors. Handling them uniformly produces systems that are simultaneously harder to build and less faithful to user intent.

This paper introduces eventual coherence, a synchronization model built on the explicit categorical separation of these operation classes. Control plane operations (CREATE, DELETE) establish resource topology. They are reconciled at synchronization boundaries, defined points at which devices exchange control plane state with the server. Data plane operations (UPDATE) modify resource content. They propagate in near-real-time throughout normal device operation. This separation is not a performance optimization. It reflects a genuine difference in the nature of the operations and their role in establishing system invariants.

Reconciliation in eventual coherence is governed by two principles. The first is local deletion finality: a device that has deleted a resource permanently ignores all subsequent data plane updates targeting that resource. The deletion is a terminal state for that device's relationship with the resource. The second is server-level upsert semantics: when a server receives a data plane update for a resource currently marked as deleted, it restores the resource. This ensures that devices with active causal participation in the resource continue to receive updates from other participants.

These two principles together resolve the delete-update conflict. The deleting device maintains its deletion. Devices that were actively using the resource continue to do so. Third-party devices receive the most recent updates. No coordination is required. No tombstones accumulate. No causal validity is violated for any participating device.

The formal contribution of this paper is the introduction of eventual coherence, a correctness criterion that operates outside the classical distributed consistency hierarchy. The traditional hierarchy is fundamentally an agreement hierarchy; it assumes that a distributed system's proximity to correctness is proportional to its total and fine-grained consensus. This assumption holds for systems modeling a single source of objective reality, such as financial ledgers. It fails catastrophically when replicas are extensions of independent human actors. A system that erases an explicit local action to force cross-replica agreement has not achieved correctness; it has achieved conformity by destroying information.

Eventual coherence rejects the assumption that agreement is the sole proxy for correctness, treating inter-replica agreement and historical validity as independent, orthogonal dimensions of a broader correctness space. This criterion is not universally appropriate. Systems where replicas must agree on a single, shared authoritative state, such as financial ledgers or inventory management, rightly prioritize consistency. Eventual coherence targets systems where devices represent independent human agents operating in distinct causal contexts, and where forcing convergence means erasing legitimate work.

This approach provides several advantages over existing methods. Unlike coordination-based systems [4, 5, 31], it operates correctly without requiring devices to communicate before performing operations. Unlike last-write-wins [10, 33], it preserves causal validity: updates performed in the context of a resource's existence are never discarded due to timestamp ordering, and deletions are never overridden by remote data plane operations on the deleting device. Unlike conflict-free replicated data types [14, 15], it requires no operational transformation, supports true deletion, and does not require unbounded storage growth. Most significantly, it achieves these properties without vector clocks, without distributed consensus, and with a protocol simple enough to implement over standard HTTP [36].

We make the following contributions:

- A formal definition of eventual coherence as a correctness criterion grounded in Lamport clock ordering, establishing its precise relationship to existing consistency models and proving it is incomparable to eventual consistency rather than merely weaker.
- A synchronization protocol that achieves eventual coherence through two principles, local deletion finality and server-level upsert semantics, requiring no physical clock synchronization, no vector clocks, and no operational transformation.
- Proofs that the protocol guarantees per-device causal validity, local deletion finality, and convergence to causally valid states within the bounds of Lamport clock ordering.
- A proof-of-concept implementation in Polyglot, a multi-device conversation management system, demonstrating practical realizability using standard web technologies, with a full reference implementation in progress.

The remainder of this paper is organized as follows. Section 2 presents our system model and formal definitions. Section 3 describes the eventual coherence protocol. Section 4 proves correctness properties. Section 5 describes the proof-of-concept implementation. Section 6 evaluates performance. Section 7 surveys related work, and Section 8 concludes.

2. SYSTEM MODEL AND DEFINITIONS

We model a distributed system consisting of a finite set of continuously connected devices $\boldsymbol{D = \{d_1, d_2, ..., d_n\}}$ that replicate a shared set of resources $\boldsymbol{R}$. Each resource $\boldsymbol{r}$ in $\boldsymbol{R}$ has a unique identifier and represents a container for mutable content (in our implementation, a conversation consisting of messages). Devices perform operations on resources and synchronize with a server $\boldsymbol{S}$ through a disciplined message passing model described below. This model follows the client-server architecture common in mobile and web applications [8, 41] rather than the peer-to-peer model of some replicated systems [42, 43].

A central premise of this model is that continuous connectivity does not imply, nor require, uniform synchronization of all operation classes. Devices deliberately propagate different classes of operations at different frequencies. This is not a concession to network unreliability. It is an architectural principle reflecting the categorical difference between the operations themselves.

2.1 Synchronization Boundaries

We introduce synchronization boundaries as a first-class concept in the model. A synchronization boundary is a point at which a device performs control plane reconciliation with $\boldsymbol{S}$. The definition of a synchronization boundary is left to the implementor: it may correspond to application launch, page load, explicit user action (such as initialization), or any other meaningful transition in device state. Between synchronization boundaries, a device operates within a local synchronization epoch, during which it processes data plane operations in near-real-time but does not pull control plane state from $\boldsymbol{S}$.

> **Definition 1 (Synchronization Epoch).** A synchronization epoch for device $\boldsymbol{d\_i}$ is the interval between two consecutive synchronization boundaries. Within a synchronization epoch, $\boldsymbol{d\_i}$ processes data plane operations in near-real-time and accumulates control plane operations for reconciliation at the next synchronization boundary.

2.2 Lamport Clocks

All causal ordering in the eventual coherence model is established via Lamport clocks [13]. No assumptions are made about physical clock synchronization across devices.

Definition 2 (Lamport Clock). Each device ***d_i*** maintains a local Lamport counter ***L_i***, initialized to 0, obeying the following rules:

- On each local operation: ***L_i := L_i + 1***
- On sending a message: the current value of ***L_i*** is included in the message.
- On receiving a message carrying Lamport value ***L_{msg}***: ***L_i := max(L_i, L_{msg}) + 1***

The server ***S*** maintains a global Lamport counter ***L_s*** updated by the same rule on every message received.

Definition 3 (Lamport Timestamp). Each operation ***o*** carries a Lamport timestamp ***L(o) = (L_i, d_i)*** where ***d_i*** is the issuing device, used for deterministic tie-breaking. The total order on Lamport timestamps is: ***(L_1, d_1)*** precedes ***(L_2, d_2)*** if ***L_1 < L_2***, or ***L_1 = L_2*** and ***d_1*** precedes ***d_2*** lexicographically. This total order is consistent with the happens-before relation [13].

Physical timestamps may be maintained for human-readable display purposes but are not load-bearing in the protocol. All ordering decisions use Lamport timestamps exclusively.

2.3 Operations

We define two categorically distinct classes of operations. This distinction is the architectural foundation of eventual coherence and reflects a genuine difference in the nature of the operations, not merely a difference in propagation timing.

Definition 4 (Control Plane Operations). A control plane operation is a function that modifies the topology of the resource set ***R***. Control plane operations are pushed to ***S*** immediately upon execution and pulled from ***S*** only at synchronization boundaries. We consider two control plane operations:

- ***CREATE(r)***: Add resource ***r*** to ***R***, with $r \notin R$ before the operation.
- ***DELETE(r)***: Mark resource $r \in R$ as deleted on the issuing device.

Definition 5 (Data Plane Operations). A data plane operation is a function that modifies the content of an existing resource without changing resource topology. Data plane operations propagate to ***S*** in near-real-time throughout a synchronization epoch.

- ***UPDATE(r, δ)***: Apply modification ***δ*** to resource $r \in R$.

Each operation ***o*** carries a Lamport timestamp ***L(o)*** as defined in Definition 3, a device identifier ***o.d***, and a resource identifier ***o.r***. This operational model differs from state-based CRDTs [14, 20], which transmit entire states, and operation-based CRDTs [15, 49], which require causal delivery [50]. Our model requires neither complete state transmission nor causal broadcast [51].

2.4 Causal History

> **Definition 6 (Local History).** The local history H_i of device d_i is the sequence of operations performed by d_i, totally ordered by Lamport timestamp $L(o)$.

> **Definition 7 (Causal Participation).** Device d_i causally participates in resource r after Lamport timestamp L if there exists an operation $o \in H_i$ such that $o.r = r$ and $L(o) > L$.

Causal participation determines how a device responds to control plane operations at synchronization boundaries. A device that has modified a resource after the Lamport timestamp of a deletion of that resource has a causal stake in the resource's continued existence. This notion tracks participation rather than complete causal history, which is both sufficient for our purposes and significantly simpler to implement than vector clock approaches [34, 35, 52].

2.5 System Assumptions

We assume an asynchronous network model [53] where message delays are unbounded but finite. Devices may crash and recover [55]. We assume crash-recovery failures only, with no Byzantine behavior [56]. S is assumed to be available and crash-recoverable with persistent storage [57].

S serves two functions. First, it maintains persistent current state for all resources, enabling devices to synchronize data plane state at any time. Second, it maintains a history of control plane events sufficient for devices to reconcile at synchronization boundaries. S broadcasts data plane updates to all connected devices in near-real-time. S does not maintain per-device state and has no knowledge of which devices have performed which local operations. S is not a source of truth for individual device state. It is a shared communication and persistence layer.

Devices maintain local replicas in persistent storage [58] and perform all operations locally without coordination [59, 60]. Data plane operations are pushed to S immediately. Control plane operations are pushed to S immediately but pulled from S only at synchronization boundaries. We make no assumption about operation ordering across devices, as in the asynchronous model of distributed computing [53, 61]. This model is weaker than systems requiring quorum reads and writes [62, 63] but stronger than systems with no server coordination [42, 64]. It reflects the practical reality of modern mobile and web applications where a shared server provides a convergence point without requiring real-time coordination for all operation classes [8, 65].

A note on scope: The model assumes data plane operations on a given resource originate from a single author operating across multiple devices. Concurrent authorship, with distinct authors independently making targeting the same resource within the same synchronization epoch, is outside the model's scope. (Under concurrent authorship, Lamport-ordered last-write-wins at the resource level discards one author's operations, violating Property 5.) Multi-author data plane semantics are left to a subsequent model.

2.6 Consistency Properties and Orthogonality

Classical distributed consistency models operate on a foundational assumption: that a system's proximity to correctness is directly proportional to its inter-replica agreement. We argue this is a framework error when replicas serve as distinct causal proxies for human intent. To evaluate such systems, we decouple correctness into two orthogonal dimensions: the Agreement Axis and the Coherence Axis. Let $\sigma(d_i, t)$ represent the materialized state of a resource set on device d_i at logical time t.

> **Definition 8 (The Agreement Axis).** A system satisfies the Agreement Axis if for any two devices $d_i, d_j \in D$, their materialized states asymptotically converge under a terminal execution trace:
>
> $$\lim_{t \to \infty} \sigma(d_i, t) = \sigma(d_j, t)$$
>
> The Agreement Axis as defined here corresponds to the standard formulation of eventual consistency [10]: absent continued updates, all replicas converge to an identical state.

> **Definition 9 (The Coherence Axis / Historical Validity).** A system satisfies the Coherence Axis for device d_i if for its local history H_i, if $o \in H_i$ is locally committed and remains within the causal horizon assumptions of the system model, then the operational mutations induced by o are deterministically preserved in $\sigma(d_i, t)$ regardless of any concurrent remote operations $o' \in H_j\ (j \neq i)$.

> **Definition 10 (Eventual Coherence).** A system provides eventual coherence if, for any resource r, when all operations on r cease and all pending synchronization boundaries have been crossed, each device's replica of r converges to a state that is causally valid given that device's local history, as ordered by the Lamport happens-before relation.

> **Proposition 1 (Orthogonality of Frameworks).** The Agreement Axis and the Coherence Axis are mutually orthogonal correctness criteria. A system may satisfy the Agreement Axis while violating the Coherence Axis on one or more devices, or conversely, satisfy the Coherence Axis on all devices while maintaining permanent state divergence ($\lim_{t \to \infty} \sigma(d_i, t) \neq \sigma(d_j, t)$).

We defer the formal proof of the Orthogonality Proposition to Section 4.5.

2.7 The Reconciliation Problem

Given this model, we state the problem eventual coherence solves.

Problem Statement (The Reconciliation Problem). Design a synchronization protocol such that:

1. Devices can perform data plane operations continuously within a synchronization epoch without coordination.
2. Control plane operations propagate to S immediately upon execution and to all devices at synchronization boundaries.
3. Data plane operations synchronize in near-real-time throughout each synchronization epoch.
4. After a synchronization boundary is crossed, each device's state is causally valid with respect to the Lamport happens-before relation.
5. A device that has deleted a resource is never required to restore it due to remote data plane operations.
6. No device loses data plane operations it performed within a synchronization epoch, provided those operations do not conflict with concurrent data plane operations from a distinct author targeting the same resource.

Traditional approaches violate at least one of these requirements. Coordination-based systems [4, 31] violate (1) due to the CAP theorem [7, 75]. Last-write-wins systems [10, 33] violate either (5) or (6) depending on timestamp ordering. CRDT systems [14, 15] make true deletion difficult, violating (6) through tombstone accumulation [21, 76] or monotonic state growth [20]. Systems using operational transformation [17, 18] achieve (5) but at significant implementation complexity [19]. Our protocol satisfies all six properties through explicit plane separation, Lamport clock ordering, local deletion finality, and server-level upsert semantics.

3. THE PROTOCOL

The eventual coherence protocol consists of four components: local operation execution, real-time data plane synchronization, real-time control plane push, and control plane reconciliation at synchronization boundaries. We present each component in turn.

3.1 Local Operation Execution

All operations execute locally without coordination [59, 77]. When device ***d_i*** performs an operation ***o*** on resource ***r***, it immediately applies the operation to its local replica, increments its Lamport counter, and records ***o*** in its local history ***H_i***. This follows the optimistic replication paradigm [2, 78].

For control plane operations:

```
CREATE(r):
    Li := Li + 1
    r.id <- generate_unique_id()
    r.lamport <- (Li, di)
    r.created_by <- di
    r.content <- empty
    local_storage.put(r)

DELETE(r):
    Li := Li + 1
    r.is_deleted <- true
    r.deleted_at_lamport <- (Li, di)
    r.deleted_by <- di
    local_storage.put(r)
```

DELETE does not remove the resource from local storage. It marks the resource as deleted and records the Lamport timestamp of the deletion for use during subsequent control plane reconciliation.

For data plane operations:

```
UPDATE(r, delta):
    if r.is_deleted then
        discard silently   // Invariant 5: Local Deletion Finality
        return
    Li := Li + 1
    r.content <- apply(r.content, delta)
    r.lamport <- (Li, di)
    r.modified_by <- di
    local_storage.put(r)
```

If a device has locally marked a resource as deleted, all subsequent data plane operations targeting that resource are silently discarded. The deletion is a terminal state for that device's relationship with the resource. This is Invariant 5 (Local Deletion Finality), stated formally in Section 3.5.

3.2 Real-Time Data Plane Synchronization

When a device performs an UPDATE operation, it immediately propagates the update to S:

```
on UPDATE(r, delta):
    apply_locally(r, delta)
    send_to_server(UPDATE_MESSAGE, r, r.lamport)
```

S receives updates, advances its global Lamport counter, and merges using Lamport-ordered last-write-wins semantics:

```
on receive UPDATE_MESSAGE(r, lamport) from device di:
    Ls := max(Ls, lamport) + 1
    r_server <- server_storage.get(r.id)

    if r_server is null then
        server_storage.put(r)
        broadcast_to_connected_devices(r)
        return

    if r_server.is_deleted then
        // Upsert: restore resource so other connected devices receive this
update.
        // Deletion record is preserved.
        // NB: correct under single-author assumption only.
        r_server.is_deleted <- false

    if r.lamport > r_server.lamport then
        r_server.content <- r.content
        r_server.lamport <- r.lamport
        server_storage.put(r_server)
        broadcast_to_connected_devices(r_server)
```

The upsert behavior is the mechanism by which third-party devices receive updates from participants who are still actively using a resource. When S receives a data plane update for a resource currently marked as deleted, it restores the resource and broadcasts the update to all connected devices. S preserves the deletion record (***deleted_at_lamport***) after the upsert. This record is required for control plane reconciliation at synchronization boundaries and is never cleared by data plane operations. Devices that have locally deleted a resource receive restoration broadcasts but discard them under Invariant 5. The broadcast reaches and correctly updates third-party devices.

3.3 Real-Time Control Plane Push

Control plane operations are pushed to S immediately upon local execution. This is the push side of the asymmetric control plane propagation model: devices push control plane events in real-time but pull them only at synchronization boundaries.

```
on CREATE(r):
    apply_locally(r)
    send_to_server(CREATE_MESSAGE, r)

on DELETE(r):
    apply_locally(r)
    send_to_server(DELETE_MESSAGE, r.id, r.deleted_at_lamport)
```

S receives control plane events and updates its persistent state:

```
on receive DELETE_MESSAGE(r_id, deleted_at_lamport) from device di:
    Ls := max(Ls, deleted_at_lamport) + 1
    r_server <- server_storage.get(r_id)
    if r_server is null then return
    r_server.deleted_at_lamport <- deleted_at_lamport
    r_server.deleted_by <- di
    r_server.is_deleted <- true
    server_storage.put(r_server)
    // Deletion is NOT broadcast to connected devices in real-time.
    // It is revealed to devices only at synchronization boundaries.
```

S does not broadcast deletions to connected devices in real-time. Deletions propagate to devices only at synchronization boundaries. This is the pull side of the asymmetric model. Data plane updates arriving after a deletion, from devices that have not yet crossed a synchronization boundary and learned of it, trigger upserts at S, restoring the resource and broadcasting updates to connected devices. Devices that have locally deleted the resource discard these broadcasts under Invariant 5.

This asymmetry has an important operational consequence: during the period between a deletion event and a device's next synchronization boundary, the server may oscillate between deleted and restored states for a resource as the deleting device's control plane push and other devices' data plane updates alternate. This oscillation is an inherent, expected architectural property of an optimistically replicated system where the server functions as a transport intermediary rather than a centralized convergence authority. It does not constitute a correctness violation. The oscillation resolves deterministically when all active participants cross their respective synchronization boundaries and commit their local participation decisions.

3.4 Control Plane Reconciliation at Synchronization Boundaries

When device ***d_i*** crosses a synchronization boundary, it reconciles its local state with S:

```
on synchronization_boundary():
    // Step 1: Advance Lamport clock from server
    Ls_current <- fetch_from_server(LAMPORT_COUNTER)
    Li := max(Li, Ls_current) + 1

    // Step 2: Fetch current server state
    server_resources <- fetch_from_server(ALL_RESOURCES)

    // Step 3: Reconcile each resource
    for each r_server in server_resources do
        r_local <- local_storage.get(r_server.id)

        // Case 1: Device has locally deleted this resource. Unconditional
finality.
        if r_local exists and r_local.is_deleted then
            send_to_server(DELETE_MESSAGE, r_local.id,
r_local.deleted_at_lamport)
            continue

        // Case 2: Evaluation of Remote Deletion Record During Client
Synchronization.
        if r_server.deleted_at_lamport is set then
            τ_local = (r_local.lamport, r_local.deviceId)  // (l_l, d_l)
            τ_delete = r_server.deleted_at_lamport          // (l_d, d_d)
            participated = r_local is not null and τ_local > τ_delete
            if not participated then
                local_storage.delete(r_server.id)
            else
                send_to_server(UPDATE_MESSAGE, r_local)

        // Case 3: No deletion record. Standard Lamport-ordered last-write-wins
merge.
        else
            if r_local is null or r_server.lamport > r_local.lamport then
                local_storage.put(r_server)

    // Step 4: Push local-only resources to server.
    local_only <- local_storage.where(lamport > last_boundary_lamport and id not
in server_resources)
    for each r in local_only do
        send_to_server(CREATE_MESSAGE, r)

    last_boundary_lamport <- Li
```

Three aspects of this algorithm require explanation. Case 1 is evaluated unconditionally before all other cases. A device that has locally deleted a resource never proceeds to the deletion record check or the LWW

merge. The device re-informs S of its local deletion, which re-asserts the deletion at S and supersedes any upsert-driven restoration. This is Local Deletion Finality in the reconciliation context.

Case 2 implements causal participation detection via full lexicographic Lamport tuple ordering. A device's local update strictly dominates the deletion record if and only if its Lamport counter exceeds the deletion's counter, or, in the equal-counter case, its device identifier is lexicographically greater than the deleting device's identifier. If the local update does not strictly dominate, including the case where logical counters are equal and the local device identifier does not dominate, the algorithm defaults to deleted. This guarantees deterministic local convergence: concurrent operations at the same logical counter always resolve in favor of the destructive control plane operation, without requiring cross-device coordination.

The default to deleted behavior for new devices falls out naturally from Case 2. A device with no local record of a resource has no causal participation. If the resource carries a deletion record, the device accepts the deletion. A new device does not acquire a resource it never participated in.

Case 3 handles resources with no deletion record through standard Lamport-ordered last-write-wins. No participation decision is required.

3.5 Correctness Invariants

> **Invariant 1 (Local Causal Consistency).** Within a single synchronization epoch on device d_i, all operations respect the happens-before relation [13]. If $o_1 \rightarrow o_2$ on d_i, then $L(o_1) < L(o_2)$ in H_i. This is guaranteed by the Lamport clock rules: each operation increments L_i before executing.

> **Invariant 2 (Control Plane Idempotence).** Applying the same control plane operation multiple times has the same effect as applying it once. CREATE is idempotent because creating an already-existing resource is a no-op. DELETE is idempotent because marking a deleted resource as deleted again changes nothing. This ensures that network failures and retries do not violate correctness [85, 86].

> **Invariant 3 (Data Plane Convergence).** If all devices cease data plane operations and all pending data plane synchronizations complete, all devices that have not deleted a resource converge to the same content for that resource. This follows from Lamport-ordered last-write-wins: the version with the highest Lamport timestamp eventually propagates to all non-deleted replicas.

> **Invariant 4 (Causal Participation Preservation).** If a device d_i executes a data plane update u on a resource R within its local history H_i, the operational intent of u is globally preserved across subsequent synchronization boundaries, provided that the device's local Lamport counter has been advanced past the deletion counter l_d associated with any remote control plane deletion of R prior to executing u.

> **Invariant 5 (Local Deletion Finality).** If device ***d_i*** has marked resource ***r*** as deleted, all subsequent data plane operations targeting ***r*** on ***d_i*** are silently discarded. Local deletion is a terminal state for a device's relationship with a resource. It cannot be overridden by remote data plane operations. This invariant applies both during normal operation (Section 3.1) and during control plane reconciliation (Section 3.4, Case 1).

3.6 Server Metadata Immutability

To guarantee that the coordination environment can accurately preserve execution history for evaluation by reconciling devices, the server must enforce absolute metadata durability for deletion records.

> **Invariant 6 (Deletion Record Timestamp Preservation).** Let ***DR*** be the server-side deletion record for a deleted resource ***R***, containing the definitive logical deletion timestamp ***τ_{deletedAt}***. From the moment of initial control plane registration until final metadata garbage collection, ***τ_{deletedAt}*** must be strictly immutable. The coordination server is explicitly prohibited from overwriting, shifting, or truncating ***τ_{deletedAt}*** in response to incoming data plane updates.

3.7 Garbage Collection

Deletion records can be garbage collected once all devices have crossed a synchronization boundary since the deletion occurred and no device has retained the resource:

```
on server_garbage_collect():
    for each r in server_storage where r.deleted_at_lamport is set do
        if all_devices_crossed_boundary_since(r.deleted_at_lamport) then
            if no_device_retained(r) then
                server_storage.delete(r.id)
```

This formulation requires S to track synchronization boundary crossings per device, which is minimal per-device state. V2 of the protocol eliminates this requirement by having devices explicitly signal completion of synchronization boundary processing, removing server-side tracking entirely. Device lifecycle presents an open problem for garbage collection. Devices that are permanently decommissioned or lost will block garbage collection indefinitely if S awaits their synchronization boundary crossing. Characterizing the interaction between device lifecycle and garbage collection is left to future work.

3.8 Complexity Analysis

Communication Complexity. Data plane updates require ***O(1)*** messages per operation (device to server, server broadcast to connected devices). Control plane push requires ***O(1)*** messages per control plane operation. Control plane reconciliation at a synchronization boundary requires ***O(|R|)*** messages, where ***|R|*** is the number of resources. Only Lamport timestamps and deletion records are exchanged during reconciliation, not complete resource state.

Storage Complexity. Each device stores $O(|R_{active}|)$ resources, where R_{active} is the subset of R not deleted from that device's perspective. S stores $O(|R|)$ resources plus $O(|R_{deleted}|)$ deletion records pending garbage collection. Unlike operation-based CRDTs [15, 49] that must store complete operation logs [88], eventual coherence requires only current state plus deletion records.

Time Complexity. Local operations complete in $O(1)$ time. Reconciliation at a synchronization boundary requires $O(|R|)$ comparisons. This is optimal: the device must examine each resource to determine its participation status.

4. CORRECTNESS PROOFS

We prove that the eventual coherence protocol satisfies its correctness properties. Proofs follow the style of Lamport [13, 69] and Lynch [32], establishing properties through invariant reasoning grounded in the Lamport happens-before relation.

4.1 Per-Device Causal Validity

We begin by establishing the primary correctness property of eventual coherence. We do not claim that the protocol satisfies classical causal consistency [39] as defined in Definition 8, which requires all devices to observe causally related operations in the same order globally. The protocol explicitly permits permanent divergence between devices with incompatible divergent causal histories. We claim instead the following.

> **Theorem 1 (Per-Device Causal Validity).** For any device d_i, the eventual coherence protocol guarantees that d_i's local state is causally valid with respect to its local history H_i at all times, including after synchronization boundary crossings.

Proof. We must show that for any operation o in H_i, the causal dependencies of o are respected in d_i's local state.

Case 1: o is a data plane operation (UPDATE). Data plane operations execute locally and immediately. By Invariant 1, operations within a synchronization epoch are totally ordered by Lamport timestamps. If o causally depends on a prior operation o' in H_i, then $L(o') < L(o)$ by the Lamport happens-before relation [13]. Since d_i applies operations in Lamport timestamp order, o' is applied before o. The causal dependency is respected.

Case 2: o is a control plane CREATE operation. CREATE establishes a resource that subsequent operations may depend on. Since CREATE executes locally and increments L_i before recording r in local storage, any operation o' in H_i that depends on r satisfies $L(o') > L(CREATE(r))$. The causal dependency is respected within H_i.

Case 3: o is a control plane DELETE operation. DELETE marks r as deleted locally. By Invariant 5 (Local Deletion Finality), no subsequent operation in H_i may target r. Therefore no operation in H_i causally depends on r existing after $L(DELETE(r))$. d_i's local state correctly reflects that r does not exist after the deletion.

Case 4: ***d_i*** crosses a synchronization boundary and discovers a deletion record for resource ***r*** at Lamport timestamp ***L_d***.

Sub-case 4a: ***r_{local}*** is null, or ***τ_{local} ⊀ τ_{delete}*** (***r_{local}***'s lexicographic timestamp does not strictly dominate the deletion record). ***d_i*** has no operation in ***H_i*** whose Lamport tuple strictly post-dates the deletion. By Definition 7, ***d_i*** did not causally participate in ***r*** after the deletion. Accepting the deletion introduces no causal violation: ***d_i*** has no operations in ***H_i*** that causally depend on ***r*** existing after the deletion record.

Sub-case 4b: ***τ_{local} ≻ τ_{delete}***. ***d_i*** has at least one operation ***o*** in ***H_i*** with ***o.r*** = ***r*** whose lexicographic Lamport tuple strictly dominates the deletion record. By Definition 7, ***d_i*** causally participated in ***r*** after the deletion. By Invariant 4 (Causal Participation Preservation), ***d_i*** retains ***r*** and ***d_i***'s operations on ***r*** after the deletion form a causally consistent sequence under the Lamport happens-before relation: each operation depends on the state of ***r*** as ***d_i*** observed it within its synchronization epoch. Since ***d_i*** did not observe the deletion before performing these operations (if ***d_i*** had, Invariant 5 would have prevented further updates), these operations are causally valid. Note that the retained operations may be concurrent with the deletion in the happens-before sense: neither ***d_i***'s updates nor the deletion causally preceded the other. The Lamport ordering establishes a consistent total order between concurrent events without implying causal precedence [13]. This is the correct treatment of concurrency under the Lamport model.

In all cases, ***d_i***'s local state respects the causal dependencies in ***H_i***. ∅

4.2 Convergence to Causally Valid States

> **Theorem 2 (Convergence).** When all operations cease and all pending synchronization boundaries have been crossed, each device converges to a state that is causally valid given its local history.

Proof. We show that the reconciliation algorithm terminates in a stable state satisfying Definition 10. Let ***T_{quiesce}*** be the time when all operations cease. Let ***T_{sync}*** be the time when all pending synchronization boundaries have been crossed. We claim that at time ***T*** > ***T_{sync}***, every device's state is causally valid.

Step 1: Reconciliation terminates. Each reconciliation processes ***O(|R|)*** resources with constant-time comparisons. Since ***|R|*** is finite and no new operations occur after ***T_{quiesce}***, reconciliation completes in finite time. Let ***T_{recon}*** be the time at which all devices complete reconciliation. ***T_{recon}*** is finite.

Step 2: The resulting state is causally valid for each device. Consider device ***d_i*** at time ***T*** > ***T_{recon}***. For each resource ***r***, exactly one of the following holds:

(a) ***d_i*** has locally deleted ***r***. By Theorem 4 (Local Deletion Stability, proved below), ***d_i***'s deletion is stable across all synchronization boundaries. ***d_i***'s state is causally valid by Case 3 of Theorem 1.

(b) ***R*** carries no deletion record and ***d_i*** holds the latest version by Lamport-ordered last-write-wins. ***d_i***'s state is causally valid by Cases 1 and 2 of Theorem 1.

(c) ***R*** carries a deletion record and ***d_i***'s local Lamport tuple for ***r*** strictly dominates the deletion record lexicographically. ***d_i*** causally participated after the deletion and retains ***r***. Thus, ***d_i***'s state is causally valid by Case 4b of Theorem 1.

(d) ***R*** carries a deletion record and ***d_i*** has no local record of ***r***, or ***d_i***'s Lamport tuple for ***r*** does not strictly dominate the deletion record. ***d_i*** accepts the deletion and thus ***d_i***'s state is causally valid by Case 4a of Theorem 1.

In all cases, ***d_i***'s final state respects all causal dependencies in ***H_i***, satisfying Definition 10. Cases (a) and (c) may produce permanently divergent states between devices. This divergence is the correct outcome under Definition 10, not a failure. Eventual coherence does not require cross-device convergence to identical state. It requires only that each device converge to a state that is causally valid for that device. ∅

4.4 Local Deletion Stability

Theorem 4 (Local Deletion Stability). If device ***d_i*** has marked resource ***r*** as deleted, then for all subsequent synchronization boundary crossings, ***d_i*** maintains ***r*** as deleted in its local state.

Proof. At each synchronization boundary crossing, the reconciliation algorithm evaluates Case 1 first and unconditionally: if ***r_{local}*** exists and ***r_{local}.is_deleted*** is true, the algorithm sends ***DELETE_MESSAGE*** to S and proceeds to the next resource without evaluating Cases 2 or 3. No path through the reconciliation algorithm sets ***r_{local}.is_deleted*** to false for a locally deleted resource. No incoming data plane update can modify a locally deleted resource (Invariant 5). Therefore once ***d_i*** has marked ***r*** as deleted, ***d_i***'s local state has ***r*** as deleted at every subsequent synchronization boundary crossing. ∅

Corollary 1. Local deletion finality is permanent absent an explicit control plane CREATE operation issued by an administrative authority with a Lamport timestamp higher than the existing deletion record. Such an operation, received by ***d_i*** at a synchronization boundary, would appear as a new resource in S's state without a deletion record post-dating the CREATE, allowing ***d_i*** to acquire the resource through Case 3 reconciliation. The issuance and authorization of such operations is outside the scope of the synchronization protocol.

4.5 Proof of the Orthogonality Proposition

We now prove the Orthogonality Proposition formulated in Section 2.6. To demonstrate that the Agreement Axis and the Coherence Axis are mutually orthogonal, we must establish two distinct conditions: (1) that a system can achieve perfect inter-replica agreement while comprehensively violating historical

validity, and (2) that a system can guarantee perfect historical validity while permitting permanent state divergence.

Lemma 1 (Agreement Without Coherence). Let A be a synchronization system whose correctness criterion requires global convergence to a single terminal state. Consider concurrent operations DELETE(r) and UPDATE(r, δ). Any deterministic reconciliation rule in A that satisfies the Agreement Axis by producing a unique, unified terminal state ($\sigma(d1) = \sigma(d2)$) must discard the structural effect of at least one committed operation. If the system converges to a deleted state, the committed update is erased; if it converges to an active state, the terminal deletion is reversed. In either execution path, system A violates the Coherence Axis for at least one device, establishing that satisfaction of the Agreement Axis does not imply satisfaction of the Coherence Axis.

Lemma 2 (Coherence Without Agreement). Let B be a synchronization system operating under the eventual coherence protocol. Consider the identical concurrent scenario: d1 issues DELETE(r) and d2 issues UPDATE(r, δ). Suppose d2's local Lamport counter has been advanced past $\tau\_delete = \langle ld, dd \rangle$ prior to issuing UPDATE(r, δ). Upon crossing a synchronization boundary, under Case 1 of the reconciliation algorithm, d1 unconditionally maintains its local deletion state ($\sigma(d1) = \varnothing$). Concurrently, under Case 2, because our setup assumption guarantees that $\tau\_local \succ \tau\_delete$, d2 retains the resource and materializes its operational update ($\sigma(d2) = \{r \mapsto \delta\}$). As $t \to \infty$, the states remain permanently diverged: $\sigma(d1) \neq \sigma(d2)$. System B violates the Agreement Axis, yet it fully satisfies the Coherence Axis for all participating nodes, as the operational mutations of both local agents are preserved.

By Lemma 1 and Lemma 2, satisfaction of one axis implies nothing about the satisfaction of the other. The Agreement Axis and the Coherence Axis are formally independent, orthogonal dimensions of distributed state evaluation.

Corollary 2 (Incomparability with Eventual Consistency). Eventual coherence and eventual consistency are incomparable: eventual consistency does not imply eventual coherence, and eventual coherence does not imply eventual consistency.

Proof. The Agreement Axis (Definition 8) is the formal expression of eventual consistency within this model. By Lemma 1, a system satisfying the Agreement Axis — and therefore eventual consistency — may fully violate the Coherence Axis; eventual consistency does not imply eventual coherence. By Lemma 2, a system satisfying the Coherence Axis on all devices may maintain permanent state divergence, violating the Agreement Axis and therefore eventual consistency; eventual coherence does not imply eventual consistency. Since neither property implies the other, the two are incomparable. ∅

4.6 Liveness

> **Theorem 5 (Liveness).** If all messages are eventually delivered and all devices eventually cross synchronization boundaries, the system reaches a stable state in finite time.

Proof. Assume eventual message delivery and eventual synchronization boundary crossings. We show that the system reaches a fixed point. Let T_quiesce be the time when all operations cease. After T_quiesce, no new operations enter any device's local history. Each subsequent synchronization boundary crossing makes deterministic decisions based on Lamport timestamps and local history. Since no new operations occur after T_quiesce, these inputs do not change. By Invariant 2 (Control Plane Idempotence), repeated synchronization boundary crossings after T_quiesce produce identical decisions. By Theorem 4 (Local Deletion Stability), deletion decisions made at synchronization boundaries are stable. By Invariant 3 (Data Plane Convergence), data plane state converges to the version with the highest Lamport timestamp among non-deleted replicas. Therefore there exists a time T > T_quiesce after which no device's state changes at any synchronization boundary crossing. The system has reached a fixed point. This fixed point may include permanently divergent states between devices. A device that has deleted a resource and a device that has retained it will remain divergent indefinitely. This is a stable and correct terminal state, not a failure mode. The liveness property guarantees termination of the reconciliation process, not convergence to identical state. ∅

5. PROOF OF CONCEPT AND FORTHCOMING IMPLEMENTATION

Polyglot is an experimental proof-of-concept system that demonstrates the practical realizability of the eventual coherence protocol using standard web technologies. This implementation uses a multi-device AI conversation management system which allows a user to maintain chat histories across multiple devices and AI providers. It is not a production system and is not presented as one. Its purpose is to demonstrate that the protocol can be realized without specialized distributed systems infrastructure, and to surface engineering considerations that a full reference implementation will need to address. A full reference implementation is in progress.

5.1 System Architecture

Polyglot consists of three components. Client devices run a React-based web application that stores conversation state in IndexedDB, a browser-native key-value store. IndexedDB provides persistent storage with ACID transactions within a single device, making it suitable for maintaining local replicas. Each browser context constitutes a device in the sense of Definition 4: it maintains an independent local history, its own Lamport counter, and crosses synchronization boundaries independently. The server is a Node.js HTTP service that maintains persistent current state for all resources and broadcasts data plane updates to connected devices in near-real-time. As established in Section 2.5, S is a shared communication and persistence layer. It does not maintain per-device state and has no knowledge of which devices have performed which local operations. Protocol implementation spans both client and server, realizing the algorithms from Section 3. The client side consists of approximately 800 lines of TypeScript across three modules: indexedDbStorage.ts

(local replica management and Lamport counter persistence), conversationStateManager.ts (operation execution and synchronization boundary logic), and syncService.ts (HTTP communication).

5.2 Data Structures

Each conversation resource r is represented as:

```
interface Conversation {
    id: string;               // Unique identifier
    title: string;            // Display name
    messages: Message[];      // Content (data plane)
    // Protocol fields -- load-bearing
    lamport: number;          // Lamport timestamp of last update
    deviceId: string;         // Device that last updated
    isDeleted: boolean;       // Current deletion state
    deletedAtLamport?: number;// Lamport timestamp of deletion
                              // Preserved through upserts (Invariant 6)
    deletedBy?: string;       // Device that issued deletion
    // Display fields -- non-protocol
    createdAt: Date;          // Human-readable creation time
    lastModified: Date;       // Human-readable last modification time
}

interface Message {
    id: string;
    role: 'user' | 'assistant';
    content: string;
    lamport: number;          // Lamport timestamp of this message
}
```

Physical timestamp fields (createdAt, lastModified) are retained for human-readable display only. They are not used in any protocol decision. All ordering decisions use the lamport field exclusively. The deletedAtLamport field requires careful handling at the server: it is set when a DELETE is received and is never cleared by subsequent data plane updates (Invariant 6). This persistence is what enables correct participation decisions at synchronization boundaries even after a resource has been restored by an upsert. The Lamport counter for each device is stored persistently in a dedicated IndexedDB store separate from conversation state. This ensures the counter survives application restarts and page refreshes. A device that restarts without its Lamport counter would be unable to correctly establish happens-before ordering with prior operations.

5.3 Local Operation Execution

Operations execute immediately on the local IndexedDB replica:

```
async createConversation(
    provider: string,
    model: string
): Promise<Conversation> {
    const lamport = await this.incrementLamport();
    const conversation: Conversation = {
        id: generateId(),
        title: 'New Conversation',
        messages: [],
        lamport,
        deviceId: this.deviceId,
        isDeleted: false,
        createdAt: new Date(),
        lastModified: new Date(),
        provider,
        currentModel: model
    };
    await this.db.conversations.put(conversation);
    return conversation;
}

async deleteConversation(id: string): Promise<void> {
    const conv = await this.db.conversations.get(id);
    if (!conv) return;
    const lamport = await this.incrementLamport();
    conv.isDeleted = true;
    conv.deletedAtLamport = lamport;
    conv.deletedBy = this.deviceId;
    await this.db.conversations.put(conv);
}

async addMessage(
    conversationId: string,
    message: Omit<Message, 'lamport'>
): Promise<void> {
    const conv = await this.db.conversations.get(conversationId);
    // Invariant 5: Local Deletion Finality
    if (!conv || conv.isDeleted) {
        return;
        // Silently discard
    }
    const lamport = await this.incrementLamport();
    const stampedMessage: Message = { ...message, lamport };
    conv.messages.push(stampedMessage);
    conv.lamport = lamport;
    conv.deviceId = this.deviceId;
    conv.lastModified = new Date();
    await this.db.conversations.put(conv);
```

```
    await this.syncToServer(conv);
}

private async incrementLamport(): Promise<number> {
    const current = await this.db.lamport.get('counter') ?? 0;
    const next = current + 1;
    await this.db.lamport.put(next, 'counter');
    return next;
}
```

The silently discarded path in addMessage directly implements Invariant 5. No error is thrown. No state is modified. The update simply does not apply. This applies equally to any data plane operation targeting a locally deleted resource.

5.4 Data Plane Synchronization

Data plane updates propagate immediately to S:

```
async syncToServer(conversation: Conversation): Promise<void> {
    try {
        const response = await fetch('http://localhost:4001/pushChats', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ chats: [conversation] })
        });
        if (!response.ok) {
            this.markDirty(conversation.id);
        }
    } catch (error) {
        this.markDirty(conversation.id);
    }
}
```

The server applies Lamport-ordered last-write-wins merging with upsert semantics:

```
function addOrUpdateChats(newChats, serverLamport) {
    const chats = readChats();
    const byId = Object.fromEntries(chats.map(c => [c.id, c]));
    let updatedLamport = serverLamport;
    for (const chat of newChats) {
        // Advance server Lamport counter
        updatedLamport = Math.max(updatedLamport, chat.lamport) + 1;
        const existing = byId[chat.id];

        if (!existing) {
            byId[chat.id] = chat;
            broadcastToConnectedDevices(chat);
            continue;
        }

        if (existing.isDeleted) {
            // Upsert: restore resource for third-party device propagation.
            existing.isDeleted = false;
        }

        if (chat.lamport > existing.lamport) {
            existing.messages = chat.messages;
            existing.lamport = chat.lamport;
            existing.deviceId = chat.deviceId;
            existing.lastModified = chat.lastModified;
            byId[chat.id] = existing;
            broadcastToConnectedDevices(existing);
        }
    }

    writeChats(Object.values(byId));
    writeLamport(updatedLamport);
}
```

The invariant on deletedAtLamport is load-bearing (Invariant 6). The upsert restores the resource so connected third-party devices receive the update. But the deletion record is preserved so that devices can make correct participation decisions at their next synchronization boundary. A resource that has been upserted will have isDeleted=false and deletedAtLamport set. This combination signals to reconciling devices that a deletion occurred and that they should check their causal participation.

5.5 Control Plane Reconciliation

Control plane DELETE operations are pushed to S immediately upon local execution:

```
async pushDeletion(conversation: Conversation): Promise<void> {
    await fetch(
        `http://localhost:4001/deleteChat/${conversation.id}`,
        {
            method: 'DELETE',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                deletedAtLamport: conversation.deletedAtLamport,
                deletedBy: conversation.deviceId
            })
        }
    );
}
```

At synchronization boundaries the client executes:

```
async onSynchronizationBoundary(): Promise<void> {
    // Step 1: Advance Lamport clock from server
    const serverLamport = await this.fetchServerLamport();
    const localLamport = await this.db.lamport.get('counter') ?? 0;
    const newLamport = Math.max(localLamport, serverLamport) + 1;
    await this.db.lamport.put(newLamport, 'counter');

    // Step 2: Fetch current server state
    const response = await fetch('http://localhost:4001/fetchChats');
    const serverConversations: Conversation[] = await response.json();

    // Step 3: Reconcile each resource
    for (const serverConv of serverConversations) {
        const localConv = await this.db.conversations.get(serverConv.id);

        // Case 1: Local deletion is final.
        if (localConv?.isDeleted) {
            await this.pushDeletion(localConv);
            continue;
        }

        // Case 2: Deletion record present.
        if (serverConv.deletedAtLamport !== undefined) {
            const τ_local = { l: localConv?.lamport ?? -1, d:
localConv?.deviceId ?? '' };
            const τ_delete = { l: serverConv.deletedAtLamport, d:
serverConv.deletedBy ?? '' };
            const dominated = τ_local.l > τ_delete.l || (τ_local.l ===
τ_delete.l && τ_local.d > τ_delete.d);
            const participated = localConv !== undefined && dominated;

            if (!participated) {
                await this.db.conversations.delete(serverConv.id);
            } else {
                await this.syncToServer(localConv);
            }
            continue;
        }

        // Case 3: No deletion record.
        if (!localConv || serverConv.lamport > localConv.lamport) {
            await this.db.conversations.put(serverConv);
        }
    }

    // Step 4: Push local-only conversations to server.
    const localOnly = await this.db.conversations
        .filter(c => !serverConversations.find(s => s.id === c.id))
```

```
            .toArray();
        if (localOnly.length > 0) {
            await fetch('http://localhost:4001/pushChats', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ chats: localOnly })
            });
        }

        this.lastBoundaryLamport = newLamport;
    }
```

Case 1 is evaluated first and unconditionally, directly implementing Theorem 4 (Local Deletion Stability). The continue statement ensures no locally deleted resource proceeds to Case 2 or Case 3.

The participated condition in Case 2 directly implements Definition 7 (Causal Participation) via full lexicographic Lamport tuple comparison. The local device must strictly dominate the deletion record: either its logical counter exceeds the deletion's counter, or — at equal counters — its device identifier is lexicographically greater. Equal counters with a non-dominant device identifier default to deleted, ensuring deterministic resolution of perfectly concurrent operations in favor of the destructive control plane action. The default to deleted behavior for new devices falls out naturally: a device with no local record of a resource (localConv is undefined) cannot have participated and defaults to deleted.

5.6 Engineering Considerations

Several implementation details required careful attention and will inform the forthcoming reference implementation.

Lamport counter persistence. The Lamport counter must survive application restarts and page refreshes. Storing it in a dedicated IndexedDB store separate from conversation state ensures it is not accidentally cleared when conversations are modified. The counter must also be advanced correctly on server state fetch: receiving the server's current Lamport value and taking the maximum before incrementing ensures the device's counter reflects the global state of the system.

Lamport counter and device identity. Each device requires a stable, unique identifier to serve as the tie-breaker in Lamport timestamp ordering. In the proof of concept this is generated on first launch and stored in localStorage. A production implementation should use a more robust identity mechanism.

Dirty state tracking. When a data plane synchronization fails, conversations are marked dirty via a separate IndexedDB index. On the next synchronization boundary, dirty conversations are pushed to S before the reconciliation fetch, ensuring that local data plane state is reflected in S before reconciliation decisions are made.

Concurrent local contexts. Multiple browser tabs on the same device share a single IndexedDB instance, including the Lamport counter store. IndexedDB transactions serialize access to the counter, preventing duplicate Lamport values within a single device. However, the reconciliation algorithm assumes a

single active context per device at each synchronization boundary. Production implementations should define explicit device boundary semantics for multi-tab scenarios.

Deletion record preservation. The most operationally critical implementation detail, formalized as Invariant 6, is that the server must never clear deletedAtLamport when applying a data plane upsert. This field is the mechanism by which reconciling devices learn that a deletion occurred and determine their causal participation. Its accidental omission would cause devices to treat upserted resources as never-deleted, preventing correct participation decisions.

Synchronization boundary definition. The proof of concept treats application initialization as a synchronization boundary. This is one valid choice among many. Production implementations might define additional synchronization boundaries at explicit user actions, periodic intervals, or application-defined state transitions. The protocol is agnostic to this choice provided synchronization boundaries occur with sufficient frequency to bound the staleness of control plane state.

5.7 Code Availability

The Polyglot proof-of-concept implementation is available at https://github.com/ForestMars/Polyglut under the MIT license. The repository is under active development. A full reference implementation conforming to the formal model of Section 2 is in progress and will be made available at the same location.

6. EVALUATION

We evaluate the eventual coherence protocol along two dimensions: correctness under the conflict scenarios its formal model addresses, and a preliminary characterization of performance properties. We do not claim benchmark superiority over alternative systems. The proof-of-concept nature of the Polyglot implementation precludes rigorous comparative evaluation. What we demonstrate is that the protocol behaves correctly under its target scenarios and that its performance characteristics are consistent with the complexity analysis of Section 3.8.

6.1 Experimental Setup

Our experiments use three simulated devices, implemented as separate browser contexts sharing no local state, communicating with a single server instance. A fourth context is used for new device experiments. Each browser context maintains an independent IndexedDB instance, its own Lamport counter, and crosses synchronization boundaries independently. Synchronization boundaries are triggered manually to allow precise control over the sequencing of conflict scenarios. We report Lamport timestamps alongside wall clock times where relevant. Lamport timestamps are the load-bearing ordering mechanism in the protocol. Wall clock times are provided for readability only.

All experiments run on a single machine with an Intel Core i7 processor and 16GB RAM. The client runs in Chrome 122 and the server runs Bun 1.3.2. Each experiment is repeated 10 times and we report median values.

6.2 Correctness Evaluation

We validate the protocol's correctness properties through systematic testing of the scenarios addressed by the formal model. Each experiment targets one or more theorems or invariants from Section 4.

Experiment 1: Core scenario — delete, update, and third-party propagation. Device A deletes conversation C at Lamport timestamp Ld=50. A pushes the deletion to the server immediately. Device B, operating within its current synchronization epoch and unaware of the deletion, adds a message to C. B's Lamport clock, advanced by prior activity on other resources, is at 80. B's update carries Lamport timestamp Lu=81. The server receives B's update for the deleted C, applies the upsert (restoring C, preserving deletedAtLamport=50), and broadcasts the restored C to Device C. Both A and B then cross synchronization boundaries.

Result. Device C receives B's update via the upsert broadcast. Device B crosses its synchronization boundary; $\tau\_local=(81, B) \succ \tau\_delete=(50, A)$, so B retains C through causal participation. Device A crosses its synchronization boundary; Case 1 triggers (A has C locally deleted), A re-asserts the deletion to the server, and A maintains C as deleted. This confirms Theorem 2 and validates the upsert propagation mechanism: third-party devices receive updates from active participants regardless of deletion state.

Experiment 2: Local Deletion Finality under upsert pressure. Device A deletes conversation C at Lamport timestamp Ld=50. Device B updates C repeatedly at Lamport timestamps 60, 70, and 80, triggering three successive upserts at the server and three restoration broadcasts to A.

Result. A discards all three restoration broadcasts under Invariant 5. A's local state has C as deleted throughout. At A's next synchronization boundary, Case 1 triggers, A re-asserts the deletion, and the server marks C as deleted again. This confirms Theorem 4: A's deletion is stable across all synchronization boundary crossings and cannot be overridden by remote data plane operations regardless of their volume or Lamport values.

Experiment 3: Default to deleted for new device. Device A deletes conversation C at Lamport timestamp Ld=50. Device B updates C, triggering a upsert at the server (C restored, deletedAtLamport=50 preserved). New Device D connects and crosses its first synchronization boundary. D has no local record of C and no causal participation.

Result. D's reconciliation algorithm reaches Case 2 for C: deletedAtLamport=50 is set and r_local is null, so the participation check fails (no strict lexicographic dominance possible with no local state). D defaults to deleted and does not acquire C. This confirms the default-to-deleted rule: a device with no causal participation accepts the deletion regardless of the server's current restoration state.

Experiment 4: Lamport participation threshold, including equal-counter tie-break. Three sub-experiments establish the boundary of the participation check, including the tie-break behavior introduced by the full lexicographic comparison.

- Sub-experiment 4a (participation confirmed). Device B has been active across multiple conversations, advancing B's Lamport clock to 120. Device A deletes conversation C at Lamport timestamp Ld=100.

B updates C at Lamport timestamp Lu=121. At B's synchronization boundary: τ_local=(121, B) $\succ$ τ_delete=(100, A). B retains C.

- Sub-experiment 4b (non-participation confirmed). Device B has been inactive. B's Lamport clock is at 40. Device A deletes C at Lamport timestamp Ld=100. B updates C at Lamport timestamp Lu=41. At B's synchronization boundary: τ_local=(41, B) $\not\succ$ τ_delete=(100, A). B accepts the deletion.
- Sub-experiment 4c (equal-counter tie-break, defaults to deleted). Device B updates C with Lamport counter l=75. Device A deletes C with Lamport counter l=75. At B's synchronization boundary: l_local = l_delete = 75. Since B < A lexicographically (B ≤ A), τ_local $\not\succ$ τ_delete, and B defaults to deleted. This confirms deterministic resolution of perfectly concurrent operations in favor of the destructive control plane operation, without cross-device coordination.

Result. The full lexicographic Lamport participation check correctly distinguishes causal participation from non-participation and resolves equal-counter ties deterministically. This validates Definition 7 as implemented.

Experiment 5: Control plane boundary delay. Device A creates conversation X within its synchronization epoch. Device B creates conversation Y within its synchronization epoch. Neither has crossed a synchronization boundary. Both then cross synchronization boundaries.

Result. Both X and Y appear on both devices after synchronization boundary crossings. New resources created within a synchronization epoch propagate correctly at the boundary through Step 4 of the reconciliation algorithm. This confirms property (1) of the Problem Statement in Section 2.7: devices perform operations continuously within epochs without coordination. These five experiments validate the protocol's correctness properties: per-device causal validity (Theorem 1), convergence to causally valid states (Theorem 2), update preservation (Theorem 3), local deletion stability (Theorem 4), and the default-to-deleted rule.

6.3 Performance Characteristics

We report preliminary performance observations consistent with the complexity analysis of Section 3.8. These are not rigorous benchmarks.

Lamport counter overhead. Lamport counter increment and persistence (IndexedDB write) adds under 1ms per operation. This overhead is negligible relative to network round-trip time and does not materially affect data plane latency.

Latency. Data plane updates complete locally in under 5ms and propagate to the server in 100 to 150ms over a local network connection, dominated by network round-trip time. Control plane reconciliation at a synchronization boundary takes 50 to 200ms depending on the number of conversations, consistent with the O(|R|) complexity of Section 3.8.

Throughput. The proof of concept handles in excess of 100 data plane updates per second per device without coordination bottlenecks. Lamport counter operations add no measurable throughput constraint.

Storage. In a test dataset of 137 conversations with between 5 and 50 messages each, total storage per device was 2.3MB. The lamport and deletedAtLamport fields added under 2KB of overhead across the dataset. Deletion records added less than 1KB of additional metadata.

6.4 Preliminary Comparison with Alternative Approaches

We implemented simplified versions of three alternative approaches within the same proof-of-concept harness to characterize behavioral differences. These are not rigorous comparative benchmarks. The alternative implementations are not optimized and the workload is not designed to stress them systematically.

Last-Write-Wins. We implemented a system in which DELETE is treated as a timestamped write operation competing with UPDATE on equal terms, using Lamport timestamps for ordering.

Result. In this model, deletion and update compete under the same ordering. If B's update carries a higher Lamport timestamp than A's deletion, B's update wins globally: C is restored on all devices including A. A, which explicitly deleted C, sees C restored. There is no equivalent of Invariant 5: the deleting device loses its deletion when outpaced by a more active retaining device. Users of the proof of concept reported this as confusing and contrary to expectation.

CRDT (Observed-Remove Set). We implemented deletion using an observed-remove set. Resources accumulate deletion records that are never garbage collected without external coordination.

Result. After 1000 operations including 100 deletions, storage grew to 4.1MB compared to 2.3MB for eventual coherence, with deletion metadata comprising 45% of total storage. The deletedAtLamport field in eventual coherence serves a similar bookkeeping function but is a single integer per deleted resource, eligible for garbage collection once all devices have reconciled. Eventual coherence achieves true deletion without unbounded storage growth.

Coordination-based (Two-Phase Commit). We implemented a two-phase commit protocol requiring acknowledgment from all participating devices before a deletion is finalized, with Lamport-ordered acknowledgments.

Result. When any device was slow to cross a synchronization boundary, deletions blocked indefinitely. Average deletion latency was 800ms compared to under 5ms for local deletion in eventual coherence. The system violated availability during control plane boundary delays, consistent with the CAP theorem analysis of Section 2.7.

6.5 Observational Notes

During informal use of the Polyglot proof of concept across multiple personal devices over approximately one month, no instances of data loss were observed. The synchronization boundary behavior matched the protocol's stated properties: resources modified within a synchronization epoch were retained after boundary crossings when causal participation was established, and resources not modified were correctly updated or deleted based on server state.

One interaction pattern confirmed Theorem 4 in practice. A conversation deleted on one device remained deleted on that device across multiple synchronization boundary crossings, even while another device continued adding messages to the same conversation. The retaining device continued to function normally. Third-party devices received updates from the retaining device through the upsert mechanism. The deleting device's state was unaffected throughout.

The default-to-deleted behavior for new devices was also observed. A new browser context connecting for the first time did not acquire conversations carrying deletion records, regardless of whether those conversations had been restored by another device's data plane updates.

6.6 Limitations

Scale and Causal Horizons. The protocol's reliance on scalar Lamport timestamps rather than vector clocks introduces a distinct causal horizon boundary for isolated operations. If a device executes a local update while its local logical clock is lagged relative to a remote deletion event ($l_{local} \leq l_{delete}$), the reconciliation algorithm will mathematically categorize the update as historically superseded. This represents the formal limits of the protocol's correctness framework: historical validity is guaranteed only for devices operating within the active causal horizon of the cluster. A device that fails to advance its local Lamport counter relative to the cluster's control plane mutations effectively drops out of the causal sequence, and its concurrent mutations are treated as causally inert upon reconciliation.

Network conditions. All experiments used local network conditions with round-trip times of 100 to 500ms. Wide-area network conditions with higher latency or more complex partition patterns may surface behaviors not observed in these experiments, though the protocol's correctness properties are not dependent on network latency bounds.

Workload generality. The workload is representative of conversation management but may not generalize to domains with substantially higher write contention, more frequent control plane operations, or different ratios of data plane to control plane activity.

Comparative evaluation. The alternative implementations evaluated in Section 6.4 are not optimized and the comparison is qualitative rather than quantitative. Rigorous comparative benchmarking against production-quality implementations of alternative approaches is left to future work.

7. RELATED WORK

Eventual coherence sits at the intersection of several research traditions. We survey each in turn, positioning our contribution precisely against the closest related work.

7.1 Limits of Convergent Approaches

The Lineage of Optimistic Replication and the Limits of Convergent Consensus

The foundations of modern collaborative software rest upon early optimistic replication frameworks designed to maximize availability in weakly connected environments. Chief among these is Bayou [1], which pioneered the use of per-replica operation logs, tentative executions, and background anti-entropy

protocols to resolve concurrent writes without synchronous coordination. To ensure that temporary divergence does not degrade into permanent state fracturing, Bayou relies on a designated primary site to assign a definitive, monotonically increasing commit sequence number to every write operation. Replicas continuously roll back tentative local states and replay operations in accordance with this global sequence, ensuring that all nodes eventually arrive at an identical ledger.

Eventual coherence differs fundamentally from these early optimistic replication models in its definition of distributed correctness. Bayou assumes that all replicas must ultimately converge to an identical state determined by a primary commit site, treating concurrent divergence as an expected transient state during anti-entropy and reconciliation to be eliminated through global serialization. In contrast, eventual coherence treats concurrent deletion and modification not as a conflict requiring resolution, but as an intentional bifurcation of intent. Where Bayou treats permanent divergence as a correctness failure, eventual coherence preserves the validity of both deletion and continued participation within their respective causal histories — elevating certain forms of permanent divergence to the correct outcome for human-facing systems.

This conceptual divergence manifests in the underlying mechanics of the reconciliation plane. Because Bayou's correctness criterion is identity, it enforces a total order that inevitably creates an operational winner and loser when confronting concurrent modifications to the same entity; a user's local, tentative deletion can be summarily overturned and resurrected by a subsequent commit sequence from the primary site. Eventual coherence replaces this centralized serialization with asymmetric reconciliation rules executed at synchronization boundaries via Lamport timestamps. By partitioning operations into discrete planes and enforcing Local Deletion Finality, our protocol guarantees that a device's choice to terminate its participation with a resource is structurally irreversible, allowing divergent data-plane timelines to coexist without violating causal validity.

7.2 Eventual Consistency and Large-Scale Systems

The practical systems literature on eventual consistency is anchored by Dynamo [10], which demonstrated that last-write-wins timestamp ordering and vector clock-based reconciliation could support high availability at scale while tolerating network partitions. Cassandra [70] and Riak [71] extended this work, with Riak subsequently adding CRDT support to address the semantic limitations of pure last-write-wins reconciliation. These systems treat all operations as equivalent writes resolved through timestamp ordering or causal dependency tracking. This uniform treatment is appropriate when the application domain does not distinguish between operations that establish resource existence and operations that modify resource content. Eventual coherence observes that for an important class of systems this distinction is fundamental, and that treating topology and content operations uniformly produces reconciliation semantics that are simultaneously harder to implement correctly and less faithful to user intent.

Dynamo's use of physical timestamps for last-write-wins is vulnerable to clock skew [13]. Eventual coherence uses Lamport timestamps for all ordering decisions, properly grounding the protocol in the happens-before relation and eliminating dependence on physical clock synchronization. For data plane operations, eventual coherence also uses last-write-wins, but under Lamport ordering rather than physical time, inheriting Dynamo's simplicity while correcting its formal grounding.

7.3 Conflict-Free Replicated Data Types and Operational Transformation

Conflict-Free Replicated Data Types (CRDTs) [14, 15, 16] offer a principled framework for coordination-free eventual consistency by leveraging mathematical properties that guarantee convergence regardless of network routing, delay, or operation reordering. State-based CRDTs [14, 20] evaluate state mutations as monotonic joins over a bounded join-semilattice, while operation-based CRDTs [15, 49] achieve the same end state by delivering commutative operations over a causally ordered transport [50]. The structural limitation of CRDTs when applied to the multi-device delete-update problem is not merely an engineering concern regarding state growth; it is a fundamental semantic mismatch dictated by their underlying algebraic invariants. By definition, a state-based CRDT requires that for any two concurrent states x and y, the merge operation must compute the least upper bound (the join) within a semilattice: $x \sqcup y$. This algebraic formulation enforces a critical system invariant: if a network achieves quiescence, replicas must converge to an identical, uniform state ($x \sqcup y = y \sqcup x$). Consequently, a CRDT lattice is structurally incapable of representing a permanent causal split.

When confronted with a concurrent delete-update pair, a CRDT must enforce a deterministic, global policy to maintain lattice validity, either by prioritizing deletion globally (as in standard Observed-Remove Sets [15]) or by preserving the updated value globally within a multi-value container (an MV-CRDT). Neither approach can satisfy the requirements of human-facing multi-device synchronization. If the system defaults to a “delete wins” policy, active updates performed in good faith by a user on a separate device are permanently erased. Conversely, if an MV-CRDT is employed to preserve both the deleted state and the concurrent update, the protocol merely defers the conflict to the application layer, forcing every device on the network to track, store, and eventually resolve the multi-value set. This introduces a subtle but severe violation of Local Deletion Finality (Invariant 5): a device that has explicitly deleted a resource is structurally barred from forgetting it. The deleting device is forced to maintain a persistent causal relationship with the dead resource, processing remote data-plane updates it has already opted out of, simply to prevent the global lattice from fracturing.

Furthermore, while modern CRDT literature proposes various optimizations for deletion record garbage collection and delta-state minimization [21, 76], these mitigation strategies invariably break the pure, zero-coordination deployment model. They rely on environmental assumptions such as physical clock synchronization, bounded network delivery latencies, or periodic stable-vector consensus protocols to safely purge stale metadata without risking state regression. Under the eventual coherence model, we reject both the coordination tax of garbage collection and the semantic tyranny of the join-semilattice. By decoupling the control and data planes, eventual coherence permits a radical departure from CRDT invariants: it allows the deleting device to enter a stable, terminal state where it completely purges and ignores the resource, while allowing the retaining device to preserve its data plane mutations. The system achieves local causal validity without forcing a uniform least upper bound across divergent human intents.

Operational transformation (OT) [17, 18] similarly preserves concurrent updates but requires the maintenance of complex operation logs and the evaluation of transform functions whose mathematical correctness is notoriously fragile [19]. Systems utilizing OT at scale, such as Jupiter [89], introduce significant architectural and computational complexity to resolve concurrency. Eventual coherence bypasses the algorithmic overhead of OT entirely. Rather than attempting to transform or synthesize conflicting

concurrent operations, our protocol partitions operations into planes with asymmetric reconciliation rules, reducing the complex delete-update paradox to a localized, deterministic Lamport timestamp comparison executed at synchronization boundaries.

7.4 Optimistic Replication and Mobile Synchronization

Optimistic replication [2, 78] executes operations locally without coordination and reconciles later, accepting temporary divergence in exchange for availability. This paradigm underlies eventual coherence's local operation execution model. Bayou [27] is the closest prior work in the mobile synchronization literature. Bayou introduced tentative writes that are reconciled lazily when devices synchronize, anticipating eventual coherence's synchronization boundary model. The critical difference is in reconciliation mechanism. Bayou requires a primary server to establish a commit ordering that resolves conflicts deterministically, which is a form of coordination. Eventual coherence achieves deterministic reconciliation without commit ordering: the Lamport timestamp comparison in Case 2 of the reconciliation algorithm makes a locally deterministic decision based only on the device's own history and the server's deletion record. No primary exists. No commit ordering is required.

The asymmetric control plane propagation model (immediate push, boundary-only pull) has been implemented in mobile computing systems [28, 54] and client-server synchronization protocols [8, 65], but has not previously been formalized as a consistency model with precise reconciliation semantics and provable correctness properties. The observation that devices push control plane events immediately while deferring their pull to synchronization boundaries is, to our knowledge, novel as a formal protocol element with defined correctness guarantees.

7.5 Control Plane and Data Plane Separation

While the separation of control and data planes is an established primitive in networking and distributed storage, existing architectures leverage this isolation to optimize or enforce uniform convergence. We are unaware of a prior synchronization model that utilizes plane separation to formalize intentional, permanent divergence as a valid correctness criterion for human-facing multi-device state. Software-defined networking [44] separates the control plane, which establishes network topology and routing policy, from the data plane, which forwards packets according to that policy. These planes operate at different timescales with different consistency requirements: control plane updates are relatively infrequent and may tolerate higher latency, while data plane operations must be fast and local. This is structurally identical to the distinction eventual coherence formalizes between CREATE and DELETE operations on the one hand and UPDATE operations on the other.

Burns et al. [Kubernetes] describe a related distinction between desired state, declared through control plane operations and reconciled by controllers, and runtime state, which reflects the current operational behavior of the system. Version control systems such as Git maintain a similar separation between refs, which establish the topology of the commit graph, and file contents, which are modified through commits. Eventual coherence formalizes this separation as a synchronization model for replicated state. The contribution is not the separation itself, which has clear antecedents in these systems traditions, but its formalization as a correctness criterion with precise reconciliation semantics, Lamport clock grounding, and provable invariants including Local Deletion Finality and Causal Participation Preservation.

7.6 The CAP Theorem and Availability

Brewer's CAP theorem [7], formalized by Gilbert and Lynch [75], establishes that a distributed system cannot simultaneously provide consistency, availability, and partition tolerance. Systems that require coordination for conflict resolution, including two-phase commit [4] and Paxos-based consensus [5, 6, 31], sacrifice availability under partition in exchange for consistency guarantees. Eventual coherence accepts this tradeoff explicitly, choosing availability. The contribution goes beyond simply accepting the tradeoff: the paper argues that for the class of systems it targets, consistency is not the correct objective even when achievable. The CAP theorem frames the choice as a reluctant concession to the realities of distributed systems. Eventual coherence reframes it as a principled design decision grounded in the semantics of the operations and the nature of the agents performing them. Forcing convergence between a device that has deliberately deleted a resource and a device that has retained it through active causal participation does not improve correctness. It imposes an arbitrary resolution on a situation that does not require one.

8. CONCLUSION

We have presented eventual coherence, a synchronization model for replicated state in multi-device human-facing systems. The model rests on two foundational observations. First, resource existence and resource modification are categorically different operations with different causal properties, different timing requirements, and different correct reconciliation behaviors. Handling them uniformly, as most synchronization protocols do, produces systems that are simultaneously harder to build and less faithful to user intent. Second, for systems where devices represent independent human agents operating in distinct causal contexts, convergence to identical state is not the correct correctness criterion. Per-device causal validity is.

The formal contribution of this paper is the definition of eventual coherence as a correctness criterion that operates outside the classical distributed consistency hierarchy. Rather than measuring correctness by agreement, we decouple system evaluation into two orthogonal dimensions — inter-replica agreement (the Agreement Axis) and local historical validity (the Coherence Axis) — and prove that neither implies the other. Corollary 2 establishes the direct consequence: eventual coherence and eventual consistency are incomparable, with neither property subsuming the other. We have proved that the protocol guarantees per-device causal validity (Theorem 1), convergence to causally valid states (Theorem 2), update preservation (Theorem 3), local deletion stability (Theorem 4), and liveness (Theorem 5). We have not claimed classical causal consistency, which requires global ordering properties incompatible with the intentional divergence eventual coherence permits. We have claimed something more appropriate for the systems we target: each device converges to a state that is correct given its own operational history, with causal ordering established through the happens-before relation rather than physical clock synchronization.

The practical contribution is a protocol built on two principles. Local deletion finality makes a device's deletion of a resource a permanent terminal state impervious to remote data plane operations. Server-level upsert semantics ensures that devices with active causal participation propagate their updates to third-party devices even in the presence of concurrent deletions. Together these principles resolve the delete-update conflict without coordination, without tombstones, without operational transformation, and without physical clock synchronization assumptions. The protocol is implementable over standard HTTP.

Eventual coherence is not the right model for every distributed system. Systems where replicas must agree on a single authoritative state, such as financial ledgers or inventory management, rightly prioritize consistency. The contribution is scoped specifically to systems where devices represent people with independent agency, where concurrent activity is the norm, and where forcing agreement means erasing legitimate work.

Future work falls into four areas. The forthcoming full reference implementation of Polyglot will provide a rigorous empirical foundation for the performance claims of Section 6 and a platform for comparative evaluation against optimized alternative implementations. The relationship between synchronization boundary frequency and control plane staleness warrants formal characterization: how often must boundaries be crossed to bound the divergence between devices, and what are the practical implications for implementors? The device lifecycle problem, specifically the interaction between device decommissioning and garbage collection of deletion records, requires treatment beyond what the proof-of-concept addresses. Finally, extending eventual coherence to multi-server topologies while preserving its correctness properties is an open problem: the current model assumes a single coordination server, and relaxing this assumption will require careful treatment of the server-level upsert mechanism and Lamport counter synchronization across servers.

The deeper point this paper makes is about what distributed correctness means for systems that serve people rather than machines. The classical consistency hierarchy asks whether replicas agree. Eventual coherence asks whether each replica’s state is valid for the agent it serves — and demonstrates that these are orthogonal questions, with neither implying the other. For an increasingly important class of systems, the second question is the right one to be asking.